EviRecall

Evidence-Aware Memory for Long-Horizon Multi-Agent Tasks

Research proposal v0.4 | 28 October 2026 | CSC5051 / CSC6052 / MDS5110

Project repository: github.com/SatouJotaro/evirecall

Member	Full name	Student ID	University email
A	[Member A name]	[Member A ID]	[Member A email]
B	[Member B name]	[Member B ID]	[Member B email]
C	[Member C name]	[Member C ID]	[Member C email]

Replace the blue personal-information placeholders before submission.

Project summary

We propose to extend Smurfs with evidence-aware shared memory and test whether checking recalled evidence prevents stale information from affecting later decisions. The project combines one memory intervention, controlled evidence/error experiments, and a reproducible benchmark harness. No performance improvements are claimed at this planning stage.

1 Motivation and research questions

Planning, execution, answering, and verification roles share information. Compressed memory may omit a constraint or become outdated after new evidence arrives. Reusing that memory can propagate an error across roles and subsequent tasks.

For example, after a user reduces a travel budget, later roles might reuse an old plan without checking the changed constraint. This is an illustrative scenario, not an experimental observation.

- **RQ1 - Memory utility:** Does tiered memory improve recall and subsequent task completion over recent history and ordinary retrieval?
- **RQ2 - Evidence and recovery:** Does checking evidence validity reduce stale-information reuse across roles and improve recovery after an update?
- **RQ3 - Resource trade-offs:** How do evidence budgets and task concurrency affect quality, cost, and completion time?

Current scope

- **Backbone and memory:** fix Smurfs; pilot Jev TierMem as the first memory backend.
- **Evaluation:** LongMemEval and STALE, with a small MemoryArena action pilot and MAST-informed failure annotation.
- **Deliverables:** a memory adapter, source-linked traces, error analysis, and reproducible comparisons. No parameter training is required initially.

Sections 2-3 compare research alternatives; Section 5 adds five implementation choices.

2 Multi-agent and memory selection

Selection favors task fit, inspectable interfaces, reproducibility, and three-person feasibility. Smurfs exposes evidence flow across roles; we hold it fixed and vary memory. The source review below does not imply reproduction of every candidate.

2.1 Multi-agent systems

Candidate and code	Relevant capability and trade-off	Decision
Smurfs [1]	Inspectable planning/execution/verification; persistent memory must be added.	**Selected.** Fits the evidence study.
Deep Agents / LangGraph [8]	Subagents, file/context tools, persistence; requires another orchestration stack.	**Alternative.** More stack changes.
Magentic-One [11]	Broad web/file/code roles; current AutoGen source is archived.	**Reference.** Extra tool setup.
TwinMarket [12]	Financial/social simulation with domain-specific tasks and evaluation.	**Outside scope.** Domain tasks.
EasyMED [13]	Patient simulation with medical tasks and educational evaluation.	**Outside scope.** Domain tasks.

2.2 Memory mechanisms

Mechanism and source	Relevant capability and trade-off	Decision
Recent history / BM25	Limited context and raw-chunk retrieval; no explicit supersession tracking.	**Required controls.**
Jev TierMem [3]	Summary/raw tiers, judgment service, Python interfaces; no router training.	**First backend.** Pilot service access.
TierMem paper implementation [2]	Provenance-linked tiers and trained routing add training/service setup.	**Method reference.** Not the Jev build.
CUP-Mem [10]	State revision and propagation-aware updates overlap our intervention.	**Closest comparator.** Pilot on STALE.
A-MEM [14] / Mem0 [15]	Other designs for memory organization, links, and updates.	**Related methods.** Optional comparisons.
Governed Memory / Caura [7]	Supersession and shared provenance; code differs from the paper artifact.	**Systems reference.** Service not required.

3 Dataset and benchmark selection

Memory QA, action execution, and trace analysis require different evaluations. Long histories alone do not establish multi-agent collaboration or action success.

Resource and code	What it evaluates; relevant limitation	Use in EviRecall
LongMemEval [4]	500 memory-QA questions: extraction, cross-session reasoning, updates, time, abstention.	**Primary.** Cleaned S; general QA.
STALE [10]	400 scenarios, three queries each; state resolution, premise resistance, policy adaptation.	**Primary.** Staleness / CUP-Mem.
MemoryArena [5]	Multi-session actions; travel requires an extra database. Code is marked preview.	**Action pilot.** Travel, if feasible.
MAST / MAST-Data [6]	1,600+ annotated traces; 14 failure modes. Not replayable Smurfs tasks.	**Annotation reference.** Not memory-training data.
MemoryAgentBench [9]	Incremental memory/conflict tests; some sources overlap LongMemEval.	**Fallback.** Memory QA only.
LoCoMo [17]	Long conversational memory; less direct coverage of our failure question.	**Optional.** Conversation check.

Dataset decision

Use two primary memory evaluations and one action pilot. These are not a training/test pair: no model parameters are trained. Reserve disjoint development groups for tuning. STALE uses LongMemEval-derived distractors; audit overlap and disclose dependence.

4 What we will build

A. Memory integration and one intervention

Connect Smurfs to a replaceable adapter with immutable source IDs. For authoritative updates, check summary validity, flag dependent conclusions, and recover raw evidence within budget. Preserve old records and uncertainty. Study these existing ideas at role handoffs; test implicit conflicts without gold update labels.

B. Evidence and error experiments

Use update/no-update cases to track stale reads, cross-role reuse, and recovery with a MAST-informed rubric. Keep stress-suite and official MemoryArena scores separate.

C. A shared benchmark harness

Run matched conditions with isolated stores and saved traces/costs. Recompute metrics and run a limited evidence-budget/concurrency sweep.

5 Implementation choices beyond agents, memory, and data

These choices define how the selected components work together. They are project decisions, not measured comparisons. Freeze pending choices after the feasibility pilot.

5.1 Model and provider configuration

Options: hosted API models or local model serving.

Initial direction: use a configurable API client and one fixed task/answer model across conditions. Choose provider, model/version, and spending cap after checking access, tool calls, context limits, and cost. Record Jev's separate service and scoring models. Local serving is optional.

5.2 Integration boundary

Options: a direct Python adapter or a separate MCP/HTTP service.

Selected: use Python calls to Jev's `observe()`, `add_summary()`, and `retrieve()` [3]. Keep `promote()` write-back explicit. Smurfs generates answers; extend its verifier to receive evidence. Keep a replaceable interface within one process initially.

5.3 Storage, retrieval, and memory scope

Options: Jev's local stores, an added vector database, or a shared memory service.

Selected: reuse Jev's SQLite/FTS5 raw store and file-based summaries [3]. Add source IDs, update links, and validity metadata in the adapter. Serialize writes within groups; isolate groups and conditions. Keep retrieval infrastructure fixed across the intervention comparison.

5.4 Action environment and tool access

Options: MemoryArena travel tools, browser/file workflows, or domain simulations.

Proposed: wrap one MemoryArena travel environment [5], subject to database access and a complete pilot trace. Match tools and limits across conditions. If infeasible, retain controlled cross-role tests and narrow claims; QA accuracy does not establish action success.

5.5 Evaluation, traces, and execution

Options: custom end-to-end judging and an external tracking stack, or official scorers with portable local artifacts.

Selected: official score wrappers plus B's evidence/recovery rubric and human checks. C owns a Python runner, versioned configs, JSONL events, and CSV summaries. Freeze judges and sampling; log calls and failures. Run isolated groups concurrently. Dashboards, distributed scheduling, and GPU experiments are optional.

6 Experimental design

6.1 Comparisons and ablations

- **Memory QA:** recent history, BM25 over identical raw chunks, unmodified Jev, and Jev plus our intervention. Fix the answer model; disclose additional models.
- **Closest method:** CUP-Mem on a matched STALE subset, subject to pilot feasibility. If unavailable, disclose the missing comparison and narrow claims.
- **Actions:** Smurfs without persistent memory, with Jev, and with our intervention; hold tools, observations, models, and limits constant.
- **Ablations:** disable validity filtering or raw-evidence recovery separately. Claims about multi-agent benefits require a budget-matched single-agent control.

6.2 Data isolation and experimental scale

Split by complete history, scenario, or task group. Gold answers, evidence-location labels, conflict explanations, and scoring state remain evaluator-only. Retain intended multi-session history but isolate stores across groups and conditions; fix query-level write-back policy. LongMemEval S, M, and oracle are context settings, not splits.

Pilot 10 development units per primary dataset and 3 travel groups. Provisionally target 100 held-out LongMemEval questions, 100 STALE scenarios, and 20 travel groups after cost estimation. Freeze IDs, prompts, models, judge, and budgets. Report coverage and counts; aim for three repetitions where affordable, with uncertainty grouped by history/scenario/task.

6.3 Quality, evidence, and recovery metrics

Use official scorers, category-level results, and human checks. Distinguish citation existence, evidence recall, and semantic support. For update cases:

- **Stale-use rate:** stale-dependent decisions / eligible post-update decisions.
- **Cross-role propagation rate:** update episodes with stale evidence reused by another role / update episodes.
- **Recovery rate:** corrected, successful episodes / update episodes.

Report denominators, failures, reviewer disagreement, and representative successful and failed traces.

6.4 Resource benchmark

Vary evidence budget and concurrency (1, 2, 4) separately on a fixed subset, holding tools and retry policy constant.

Count all role, summary, retrieval, Jev, and retry calls, including failed runs. Separate ingestion, execution, and evaluation-judge costs. Report tokens, requests, throughput, and latency including queue time; unknown usage is not zero. Label injected delays as synthetic. API results do not establish GPU utilization or hardware-scheduling performance.

7 Work allocation and collaboration

Each member owns implementation, experiments, analysis, and writing. Proposed boundaries keep the packages independently testable while making one integrated project.

Owner	Implementation and experiments	Report contribution
A - Memory	Memory backends; Smurfs hooks; one update intervention; mechanism ablations.	Method, integration, ablations.
B - Evidence and errors	STALE input/label contract; one travel adapter; paired cases; evidence/error analysis.	Task design, evidence analysis, qualitative results.
C - Benchmark	Runner; QA loading; configs/splits; score wrappers; usage/latency and resource study.	Shared setup, efficiency, reproducibility.

Independent and balanced work

Agree on interfaces first: A uses fixtures, B scores fixed traces, and C runs a dummy backend. Integrate in week 1 and cross-review. Each owner implements their own component.

A owns one intervention, B one action environment, and C one runner with two resource axes. Rebalance a bounded adapter or scorer after the pilot; equal effort is a planning target.

Report assembly

Each member writes their sections from one frozen protocol. Jointly edit the introduction, terminology, conclusions, limitations, and contributions. Interpret the combined results together.

8 Milestones, risks, and deliverables

Stage	Team milestone and acceptance condition
Week 1: feasibility	Agree on interfaces; pilot Smurfs/Jev, STALE/CUP-Mem, and travel. Verify access, costs, input boundaries, and a complete trace; freeze scope.
Weeks 2-3: development	Finish baselines, intervention, cases, and logging. Tune on development data; freeze test manifests and protocol.
Week 4: evaluation	Run comparisons, ablations, and resource sweeps. Retain failures, outputs, configurations, and scores.
Week 5: delivery	Cross-reproduce key results; assemble report/poster/demo; verify individual contribution claims.

These are project stages, not official deadlines. A failed action pilot limits claims to memory and controlled cross-role tests. Disclose any Jev replacement.

Deliver code/configs, cases, traces, analysis, and reproduction instructions. Submit a LaTeX report with individual contributions, GitHub source, and a poster presentation [16], following the confirmed course format. Report positive or negative results.

References

[1] Junzhi Chen, Juhao Liang, Benyou Wang. Smurfs: Multi-Agent System using Context-Efficient DFSDT for Tool Planning. 2025. NAACL, pp. 3281-3298. [Paper] [Code]

[2] Qiming Zhu, Shunian Chen, Rui Yu et al.. From Lossy to Verified: A Provenance-Aware Tiered Memory for Agents. 2026. arXiv:2602.17913. [Paper] [Code]

[3] FreedomIntelligence. Jev TierMem: Lightweight Agent Memory. 2026. Software documentation; accessed 2026-10-27. [Code]

[4] Di Wu, Hongwei Wang, Wenhao Yu et al.. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. 2025. arXiv:2410.10813. [Paper] [Code] [Data]

[5] Zexue He, Yu Wang, Churan Zhi et al.. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. 2026. arXiv:2602.16313. [Paper] [Code] [Data]

[6] Mert Cemri, Melissa Z. Pan, Shuyi Yang et al.. Why Do Multi-Agent LLM Systems Fail?. 2025. arXiv:2503.13657. [Paper] [Code] [Data]

[7] Yanki Margalit, Nurit Cohen-Inger, Erni Avram et al.. Governed Shared Memory for Multi-Agent LLM Systems. 2026. arXiv:2606.24535. [Paper] [Code]

[8] LangChain. Deep Agents Overview. 2026. Official software documentation; accessed 2026-10-27. [Docs] [Code]

[9] Yuanzhe Hu, Yu Wang, Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. 2026. arXiv:2507.05257. [Paper] [Code] [Data]

[10] Hanxiang Chao, Yihan Bai, Rui Sheng et al.. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?. 2026. arXiv:2605.06527. [Paper] [Code] [Data]

[11] Adam Fourney, Gagan Bansal, Hussein Mozannar et al.. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. 2024. arXiv:2411.04468. [Paper] [Code]

[12] Yuzhe Yang, Yifei Zhang, Minghao Wu et al.. TwinMarket: A Scalable Behavioral and Social Simulation for Financial Markets. 2025. arXiv:2502.01506. [Paper] [Code]

[13] Bingquan Zhang, Xiaoxiao Liu, Yuchi Wang et al.. Human or LLM as Standardized Patients? A Comparative Study in Medical Education. 2025. arXiv:2511.14783. [Paper] [Code]

[14] Wujiang Xu, Zujie Liang, Kai Mei et al.. A-Mem: Agentic Memory for LLM Agents. 2025. arXiv:2502.12110. [Paper] [Code]

[15] Prateek Chhikara, Dev Khant, Saket Aryan et al.. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. 2025. arXiv:2504.19413. [Paper] [Code]

[16] NLP Course, CUHK-Shenzhen. CSC5051/CSC6052/MDS5110 Natural Language Processing, Fall 2026. 2026. Course website; accessed 2026-10-27. [Source]

[17] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov et al.. Evaluating Very Long-Term Conversational Memory of LLM Agents. 2024. arXiv:2402.17753. [Paper] [Code]